

Texas A&M University
CSCE 703 — Cybersecurity

Homework 2B

OWASP Juice Shop: Secure Design, Login Form, and Exploitation

Name	Siva Sai Deepank Manoj
UIN	437005609
Email	deeps45@tamu.edu
Course	CSCE 703
GitHub	https://github.com/deeps45/juice-guard

Part 1 — Secure Feature Design

I exploited three Juice Shop issues on preview.owasp-juice.shop and used those results to design a safer registration/login flow. **Login Admin (Injection):** I submitted the email ' OR 1=1 -- with any password. The string-built SQL login query treated that as code, so the API returned HTTP 200 with a JWT for the first user (admin role). Observed result: Account menu showed `attacker-owned@x.io`, and the Score Board marked Login Admin solved (green). Mitigation: parameterized queries/ORM, validate email before the DB call, and never concatenate user input into SQL. **DOM XSS:** Navigating to `/#/search?q=<iframe src="javascript:alert(`xss`)">` executed the payload in the search DOM. Observed result: the app's own “DOM XSS fired — `alert("xss")`” banner, and the DOM XSS card turned green. Mitigation: encode output, prefer `textContent` over `innerHTML`, and ship a CSP. **Unsigned / forged JWT:** Weak JWT handling lets an attacker mint tokens (`alg:none` / `public-key-as-HMAC`). Observed result: Unsigned JWT solved; `GET /rest/user/whoami` accepted a crafted token. Mitigation: verify signatures with the private key only, short expiry, and server-side authorization. Registration passwords must be stored with `bcrypt` (cost 12), never plaintext.

Learnings (~100 words)

Actually running the payloads changed how I think about “known vulns.” Seeing ' OR 1=1-- return an admin JWT, and watching the DOM XSS banner fire, made the risk concrete: one bad concat or one `innerHTML` sink is enough. JWT handling mattered just as much as passwords—token acceptance is another login path. My design takeaway is short: treat every field as hostile, keep queries parameterized, never render raw HTML from users, verify tokens correctly, and hash passwords with `bcrypt` before storage. Those habits map directly onto the three breaks I documented with input and observed result.

Password hashing example

```
const hash = await bcrypt.hash(password, 12);
const ok = await bcrypt.compare(submittedPassword, user.passwordHash);
```

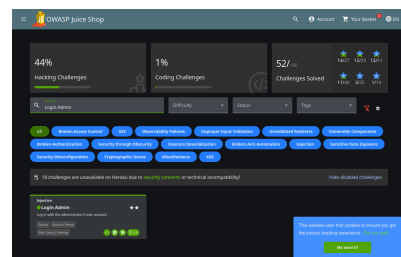
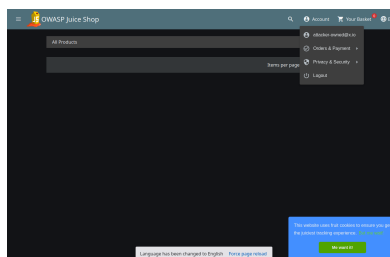
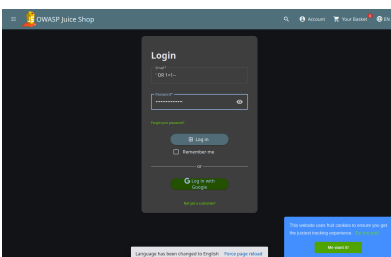


Figure 1. Login Admin input: ' OR 1=1--.

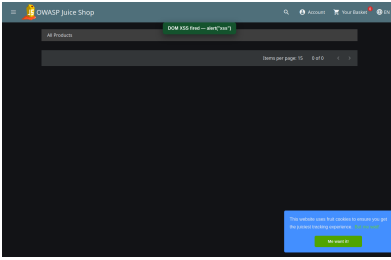


Figure 4. DOM XSS fired (alert banner).

Figure 2. Result: logged in as attacker-owned@x.io.

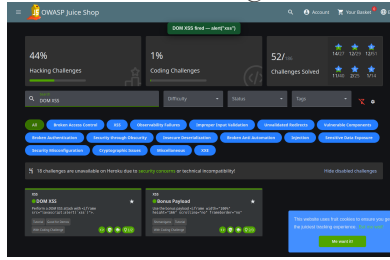


Figure 5. DOM XSS solved (green).

Figure 3. Login Admin solved (green).

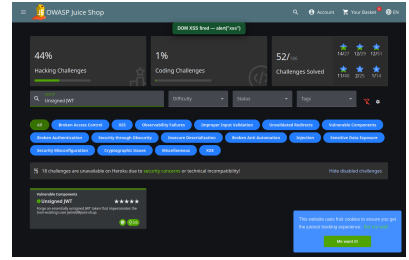


Figure 6. Unsigned JWT solved (green).

Part 2 — Front-End Login Form

I built a hardened login page (HTML/CSS/JS + Express) in the public repo. Client-side, `app.js` blocks empty submit, requires “@” in the email, and requires password length ≥ 8 . Server-side, those checks run again more strictly; passwords are verified with `bcrypt.compare` at cost 12 against a real hash for every attempt (unknown emails use a precomputed dummy hash so timing does not leak account existence). Successful login sets an `httpOnly session` cookie; register never returns 409 for an existing email. Login also requires a CSRF token from `/api/csrf`. Status messages use `textContent`. Headers include CSP, X-Frame-Options: DENY, nosniff, and rate limiting. The process binds `0.0.0.0:3847` and logs that same address.

Learnings (~100 words)

Building the form made the Juice Shop lessons practical. Client checks are UX—easy to skip—so the server is the real gate. CSRF forced me to think about forged requests, not only bad passwords. Using a dummy `bcrypt` hash for unknown emails closed a timing leak I had left earlier; uniform register responses closed enumeration. Hashing with `bcrypt` was straightforward once I stopped thinking “encryption” and started thinking one-way hashing. A basic login still needs validation, safe rendering, hashing, sessions, and request authenticity. The README documents how to run the hardened form and the vulnerable lab locally.

Public GitHub repository: <https://github.com/deeps45/juice-guard>

Run: `npm install && npm start` → `http://127.0.0.1:3847`

Demo login (README only): `demo@juice.shop` / `JuiceShop1!`

Vulnerable lab (Part 3 replay): `http://127.0.0.1:3847/vulnerable/`

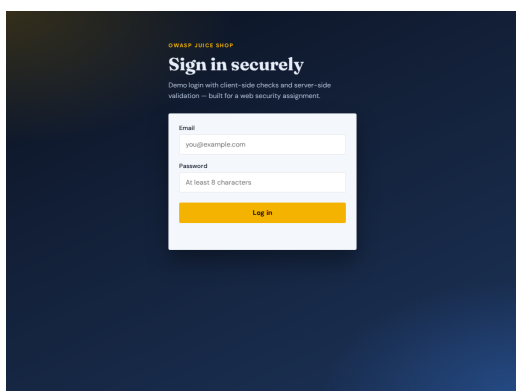


Figure 7. Hardened login form.

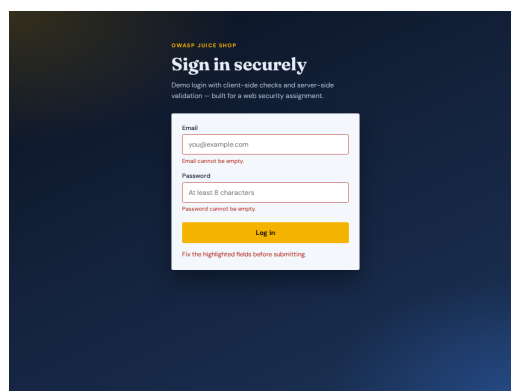


Figure 8. Empty fields blocked.

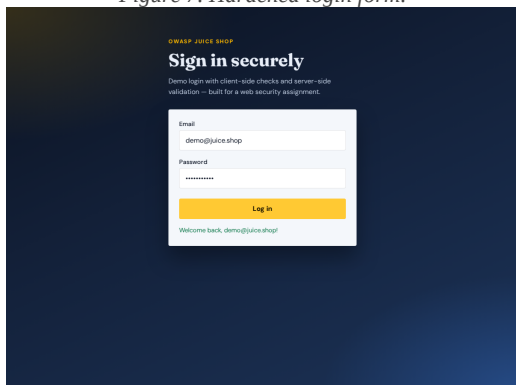


Figure 9. Successful login + session.

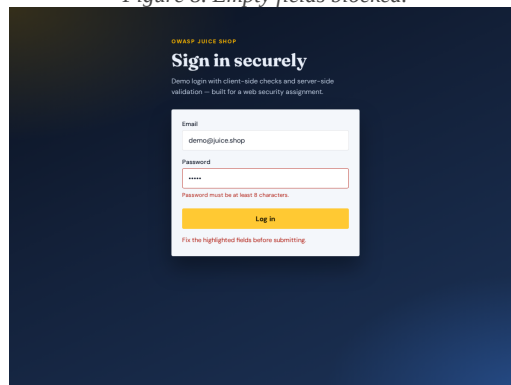


Figure 10. Short password rejected.

Part 3 — Breaking My Own Form

Part 3 needs a payload that actually runs, not only controls that reject input. I kept a replayable vulnerable lab at `/vulnerable/`: no CSP, and after submit it does `reflected.innerHTML = "Submitted email: " + email` (same class of bug as Juice Shop DOM XSS). **Successful exploit:** open `/vulnerable/`, email `<img src=x onerror="alert('XSS')">@evil.com`, password `password123`, Log in. Observed result: the browser alert fired, and the page rendered XSS EXECUTED via `innerHTML` in red—script ran from the reflected email. Graders can replay this from the repo without guessing. On the hardened form (`/`), the same payload does not execute: email validation rejects unsafe characters, status uses `textContent`, and CSP blocks inline handlers. A script-shaped password still posts past the `length/@` checks, but the server returns 400; a login without X-CSRF-Token returns 403. SQLi-shaped emails fail because this app has no SQL and validates addresses—not because a quote denylist is the boundary. XSS fails on the hardened path because of `textContent` + CSP, not because “`<`” is banned in the password alone.

Learnings (~100 words)

The useful lesson was watching a real break, then turning it off. Leaving `innerHTML` on the lab page made XSS obvious; switching to `textContent` and CSP on the hardened page made the fix equally obvious. Client validation still let a script-shaped password reach the API, which is why the server must be the boundary. CSRF testing showed why state-changing requests need a token. I will design the server path first, keep a deliberate vulnerable demo only when I need to prove an exploit, and never treat the browser as security by itself again.

Steps and results (replayable)

1. **Successful XSS (vulnerable lab):** `npm start` → open `/vulnerable/` → email `<img src=x onerror="alert('XSS')">@evil.com` → password `≥ 8 chars` → Log in. *Result:* `alert('XSS');` DOM shows “XSS EXECUTED via `innerHTML`”.

2. **Same payload on hardened `/`:** *Result:* no alert; server/client reject unsafe email; `textContent` + CSP.

3. **Client bypass:** `fetch('/api/login')` with CSRF + `<script>` password. *Result:* HTTP 400 from server.

4. **CSRF:** POST valid credentials with no X-CSRF-Token. *Result:* HTTP 403.

Fix applied: hardened `/` uses `textContent`, CSP, CSRF, `bcrypt` (always), sessions, and non-enumerating register; `/vulnerable/` remains for grader replay.

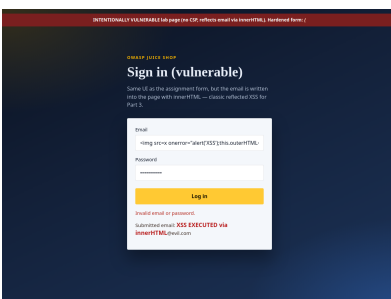


Figure 11. XSS succeeds on `/vulnerable/`.

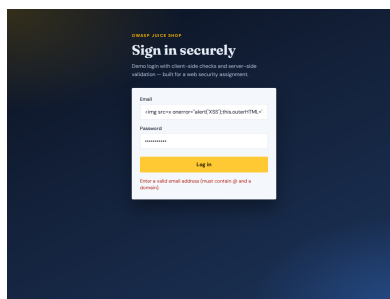


Figure 12. Same payload blocked on `/`.



Figure 13. CSRF 403 + `fetch` 400 (browser).

Mechanism note: the working XSS is reflected DOM XSS via innerHTML on the lab page. The hardened app's SQLi probes fail because there is no SQL engine and emails are validated as data—not because quoting is “banned” as a primary control.